

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

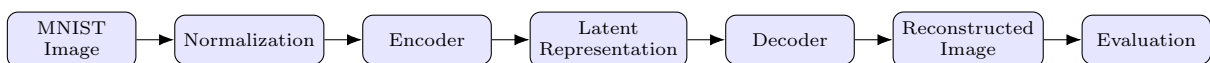
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

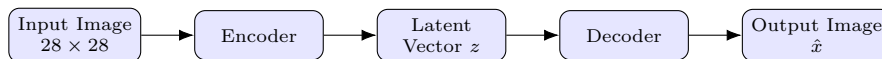
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

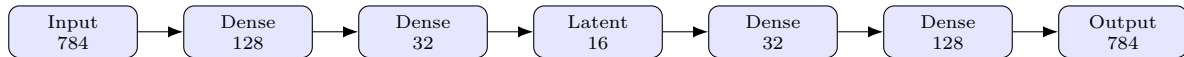
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

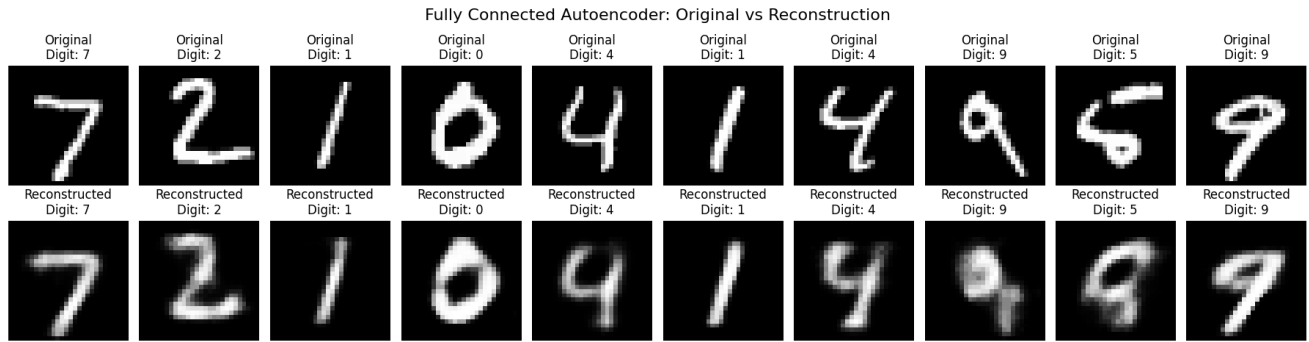


Figure 1: Fully connected autoencoder: original test images (top) and reconstructions (bottom).

Observed result: The reconstruction preserves the broad identity of the digits, but thin strokes and fine boundaries are visibly smoothed. This is consistent with the fully connected architecture, which operates on flattened pixels rather than explicitly preserving local spatial neighborhoods.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

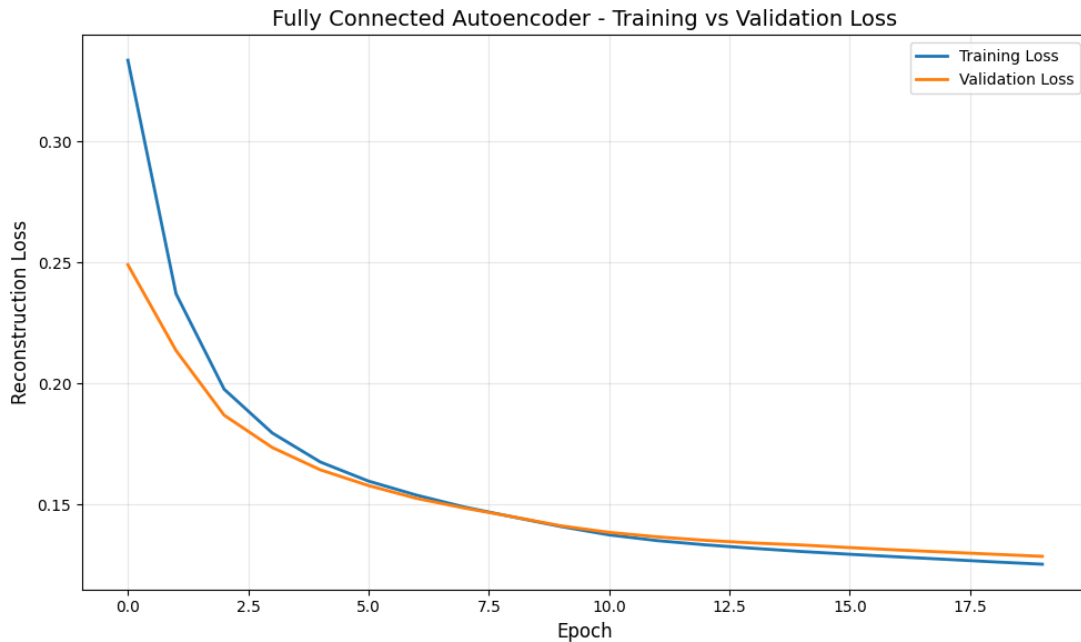


Figure 2: Fully connected autoencoder training and validation loss.

Notebook result: Training loss decreased from 0.333636 to 0.125405, while validation loss decreased from 0.249100 to 0.128652 over 20 epochs. The curves move downward and remain relatively close near convergence.

Required inference: The model converged during the observed run. The notebook reports that training and validation losses were relatively close, with no strong evidence of severe overfitting in the final epochs.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

Report:

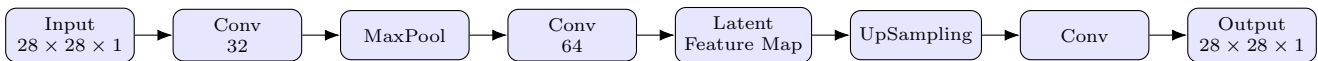
Metric	Value
Test MSE	0.022515
Test MAE	0.059259
Mean SSIM	0.734634

The notebook also reports a per-image MSE range of 0.001632 to 0.070605, with median 0.021362. The five largest-error samples were test indices 1782 (digit 8), 1017 (digit 6), 1758 (digit 8), 1170 (digit 8), and 654 (digit 5).

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022515	0.059259	0.734634	211,040
Convolutional AE	0.002537	0.014808	0.974641	74,497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.

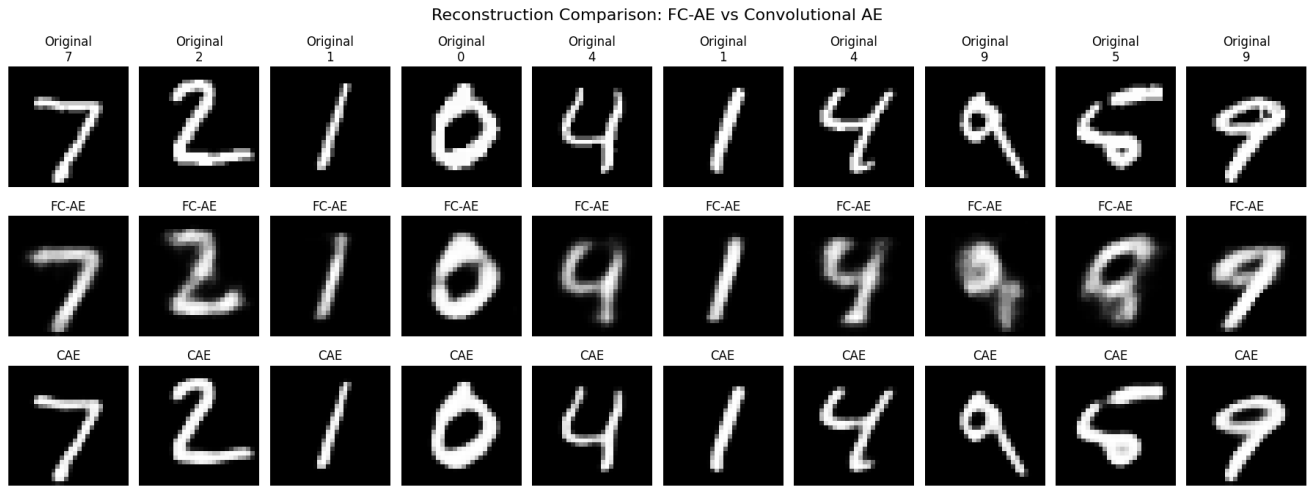


Figure 3: Side-by-side comparison of original, fully connected AE, and convolutional AE reconstructions.

Observed result: In the executed notebook, the convolutional autoencoder achieved substantially lower reconstruction error and higher SSIM while using fewer parameters. The visual comparison also shows better preservation of digit strokes and local structure. This is consistent with convolutional layers retaining spatial locality.

Required inference:

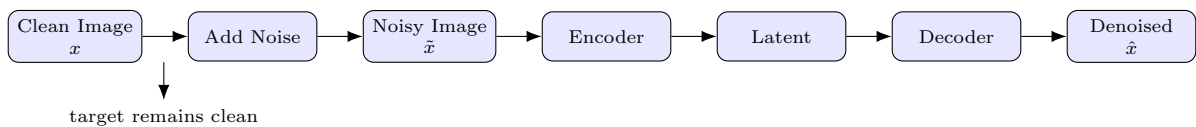
The final recovery run reports FC-AE $MSE = 0.022515$, whereas the CAE reports $MSE = 0.002537$. The corresponding SSIM values are 0.734634 and 0.974641, respectively.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

The notebook additionally trained a salt-and-pepper denoising model at $p = 0.10$. Its test performance was $MSE = 0.004813$, $MAE = 0.020720$, and $SSIM = 0.946327$.

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

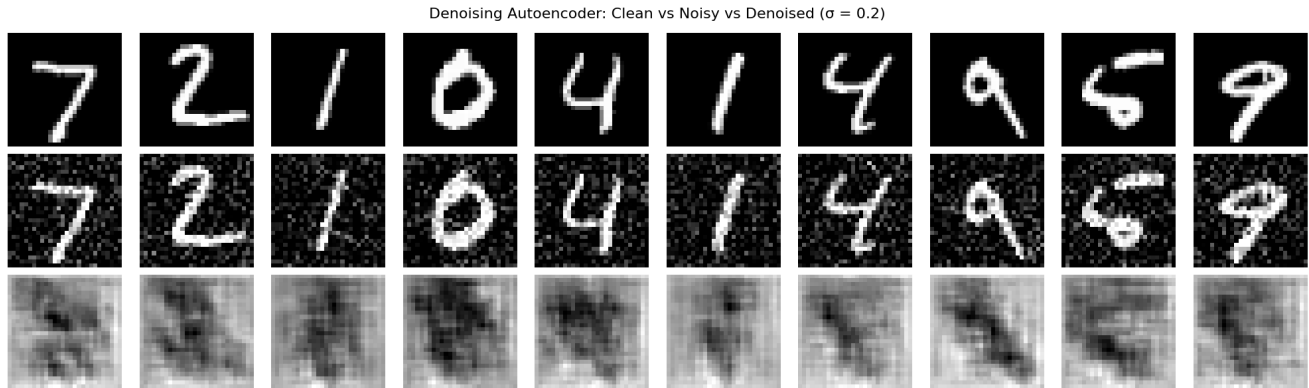


Figure 4: Gaussian denoising at $\sigma = 0.2$: clean, noisy, and denoised images.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

MSE , MAE , $SSIM$.

Report:

Gaussian Noise Level	Noisy-vs-Clean MSE	MAE	SSIM
$\sigma = 0.1$	0.228792	0.470260	-0.002904
$\sigma = 0.2$	0.227955	0.469419	-0.001729
$\sigma = 0.3$	0.227147	0.468600	-0.000658

These three values describe the corrupted images before denoising; they are therefore a measure of corruption severity rather than the final denoising model performance. The trained Gaussian denoising autoencoder used $\sigma = 0.2$ and achieved test $MSE = 0.016222$, $MAE = 0.059488$, and $SSIM = 0.642149$ in the final consolidated results.

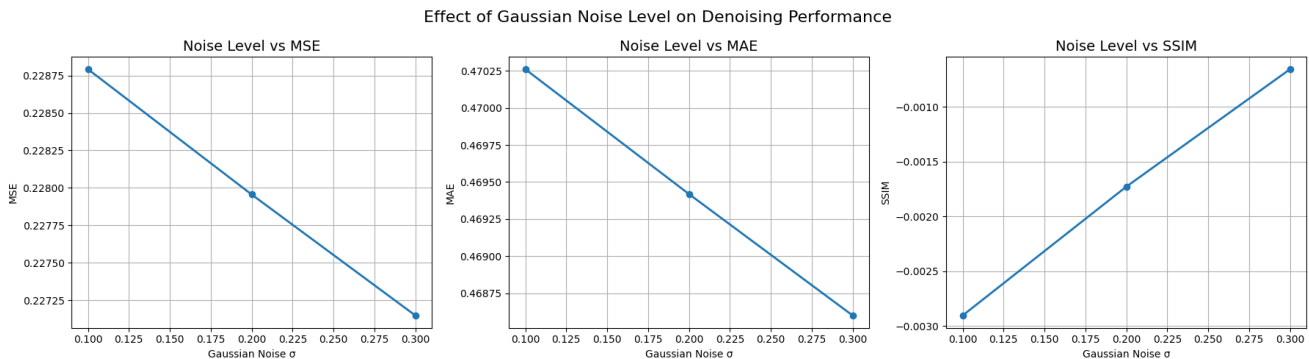


Figure 5: Effect of Gaussian noise level on corruption metrics recorded by the notebook.

The notebook trained the Gaussian denoising model for 20 epochs using noisy images with $\sigma = 0.2$ as inputs and clean images as targets. Training loss decreased from 0.074962 to 0.072421, and validation loss decreased from 0.073502 to 0.071459.

Required inference: Increasing Gaussian noise changes the input distribution substantially, while the denoising model learns to recover the clean target. The executed $\sigma = 0.2$ model reduced the test MSE to 0.016222 and produced recognizable denoised digits in the visual output.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

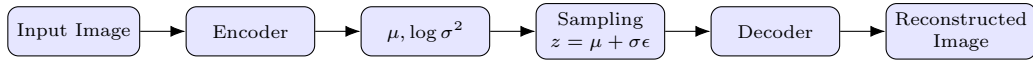
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

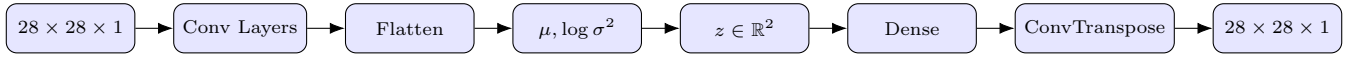
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

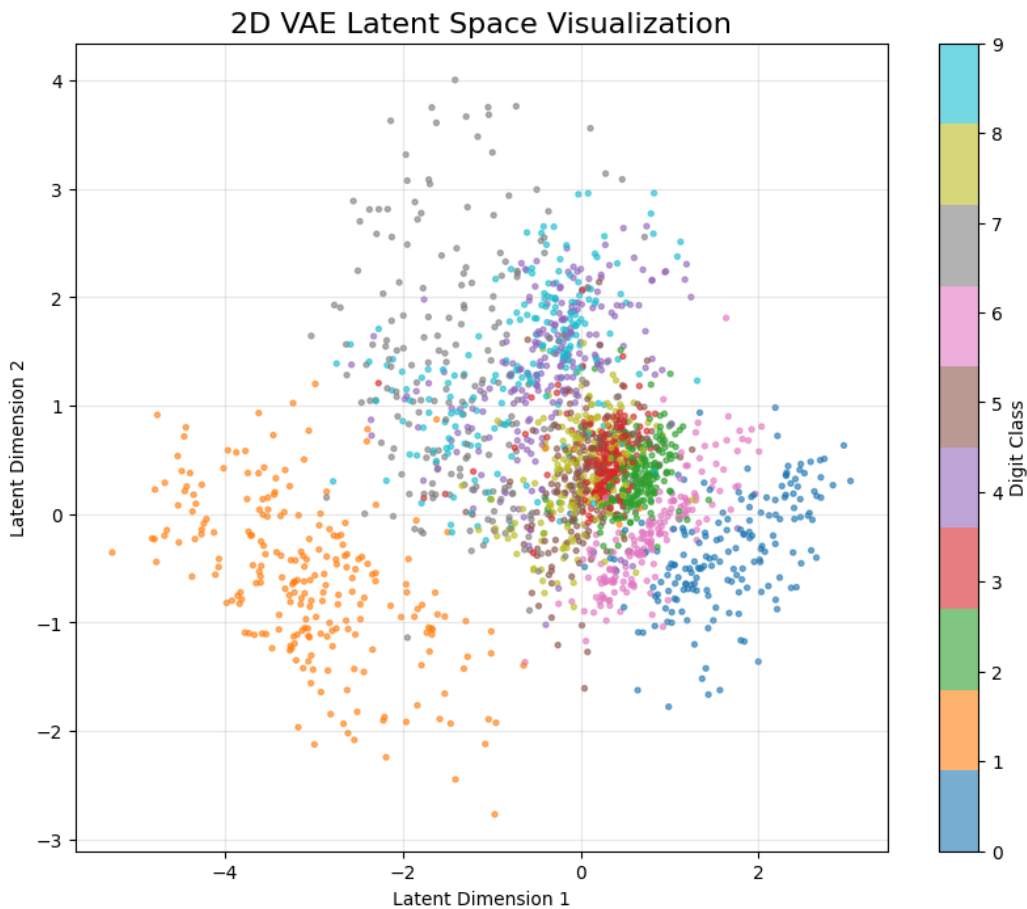


Figure 6: Two-dimensional VAE latent-space visualization for the 2,000-image test subset.

Observed result: The latent representation forms structured regions for the digit classes, but the class clouds are not perfectly separated. Several classes overlap, indicating that the two-dimensional latent bottleneck captures useful structure while imposing strong compression.

Required inference:

The VAE maps images to a probabilistic two-dimensional latent space. Similar digits tend to occupy related regions, while overlap remains because the model is simultaneously encouraged to reconstruct images and keep the latent distribution close to the standard normal prior.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

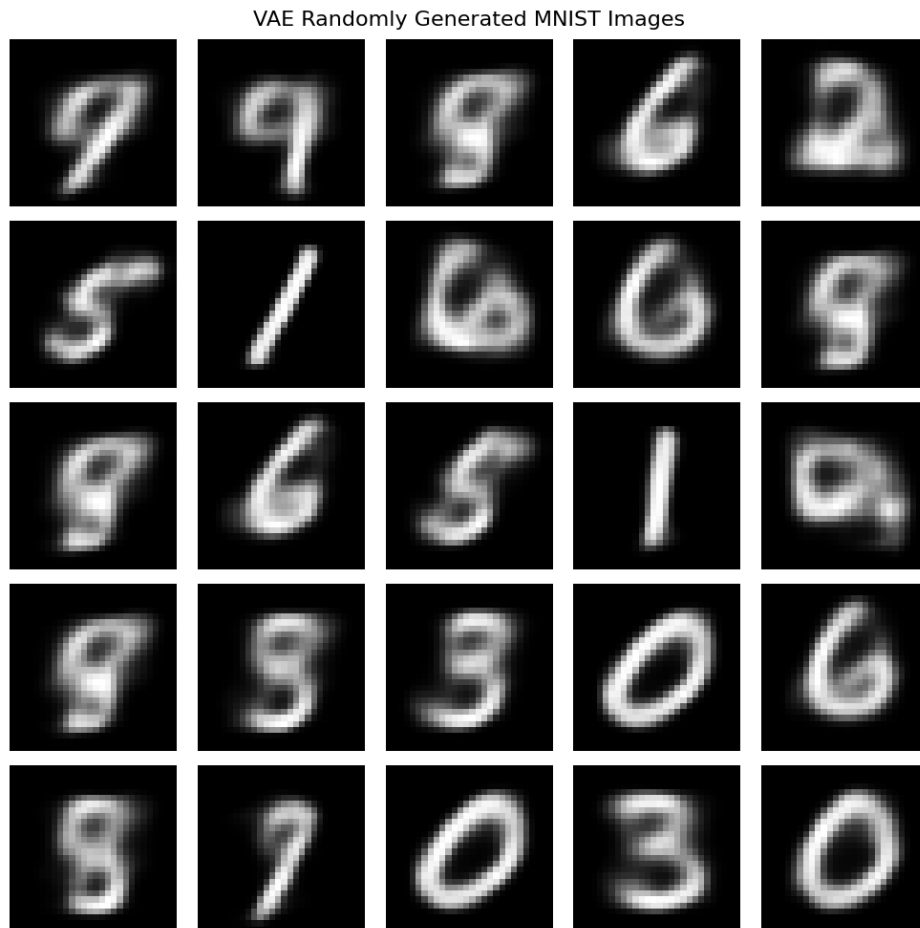


Figure 7: Twenty-five VAE samples generated from $z \sim \mathcal{N}(0, I)$.

The executed notebook generated 25 images. The samples exhibit recognizable digit-like structures with visible variation, although some samples are less sharply defined than the original MNIST images.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

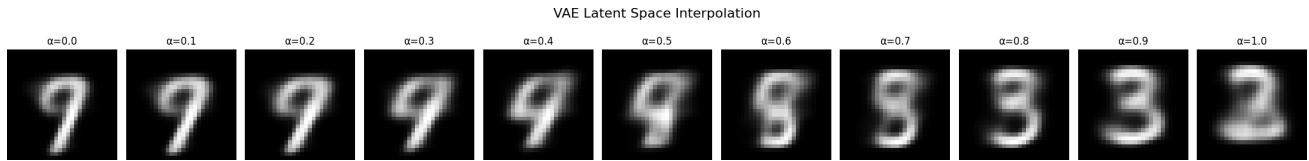


Figure 8: VAE latent-space interpolation using $\alpha = 0, 0.1, \dots, 1$.

Required inference: The interpolation output shows a gradual change in decoded appearance between the selected latent endpoints. This illustrates the continuous structure learned in the VAE latent space and is distinct from simply memorizing individual training examples.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022515	0.059259	0.734634	211,040
Convolutional AE	0.002537	0.014808	0.974641	74,497
Denoising AE ($\sigma = 0.2$)	0.016222	0.059488	0.642149	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Final validation reconstruction loss	147.6846
Final validation KL loss	6.0090
Final validation total loss	153.6936
Test MSE	0.043134
Test MAE	0.100257
Mean SSIM	0.494859
Mean KL divergence on test set	6.013857

The VAE used latent dimension 2, $\beta = 1.0$, Adam with learning rate 0.001, 20 epochs and batch size 128. The executed run took 638.37 seconds and used 485,957 parameters.

22. Required Plots

The executed notebook produced the requested visualizations. Representative figures are included throughout this report, including the FC-AE loss curve, reconstruction comparisons, denoising outputs, VAE latent space, generated samples, interpolation, reconstruction-error distribution, and final model comparison.

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

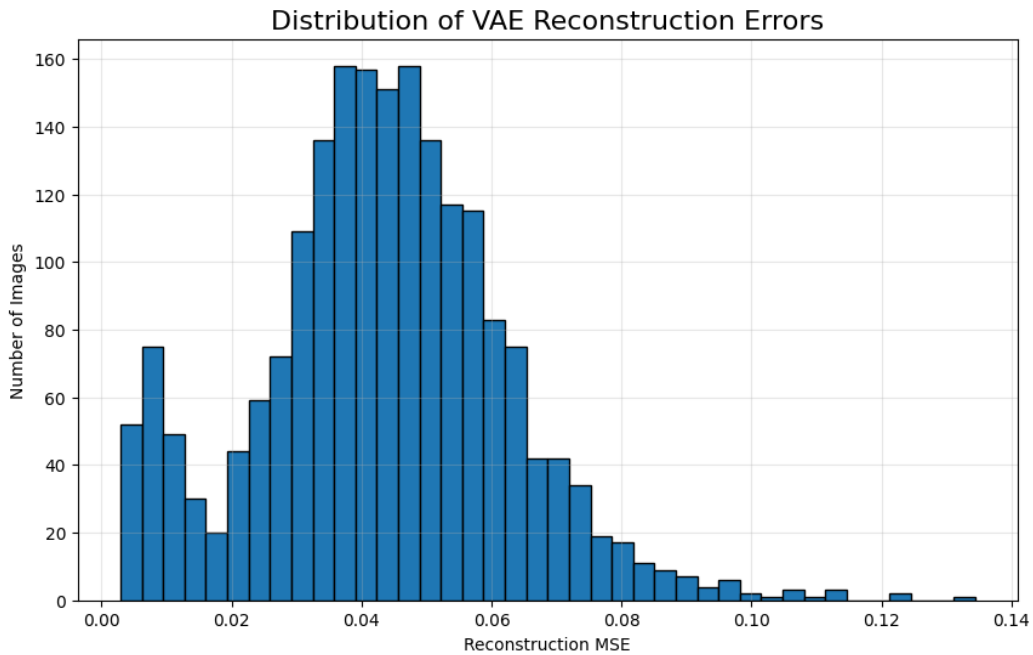


Figure 9: Distribution of VAE reconstruction errors on the test subset.

The VAE reconstruction-error distribution is concentrated around moderate errors with a visible right tail, indicating a smaller number of substantially harder samples.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

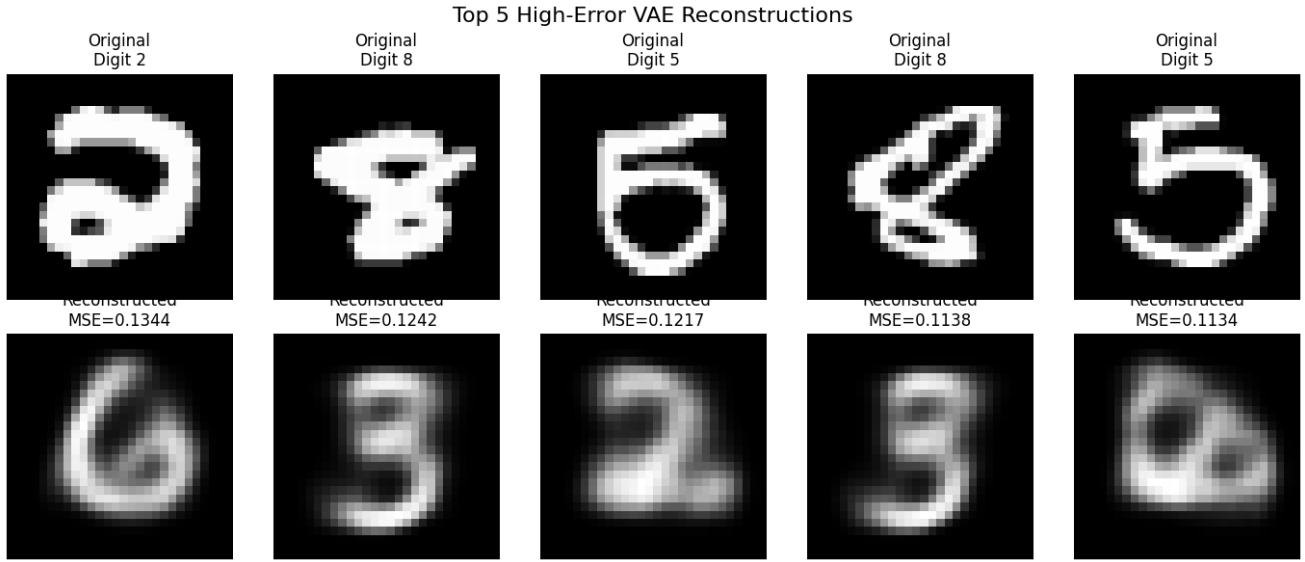


Figure 10: Five highest-error VAE reconstructions.

The notebook identifies the following five highest-error samples:

Rank	Test Index	Digit	MSE
1	1352	2	0.134376
2	998	8	0.124196
3	1847	5	0.121735
4	1325	8	0.113770
5	352	5	0.113375

These examples are difficult because the VAE's low-dimensional probabilistic bottleneck must represent stroke variations using only two latent variables, so unusual shapes or ambiguous strokes can lose detail during reconstruction. Students should provide a short explanation for why these images may be difficult to reconstruct.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.022515	0.059259	0.734634	211,040	23.57
Conv. Autoencoder	0.002537	0.014808	0.974641	74,497	699.52
Denoising AE ($\sigma = 0.2$)	0.016222	0.059488	0.642149	74,497	713.39
VAE	0.043134	0.100257	0.494859	485,957	638.37

The consolidated values above use the notebook's final recovery/consolidation outputs. The notebook also reports a salt-and-pepper denoising run at $p = 0.10$ with $MSE = 0.004813$, $MAE = 0.020720$, and $SSIM = 0.946327$.

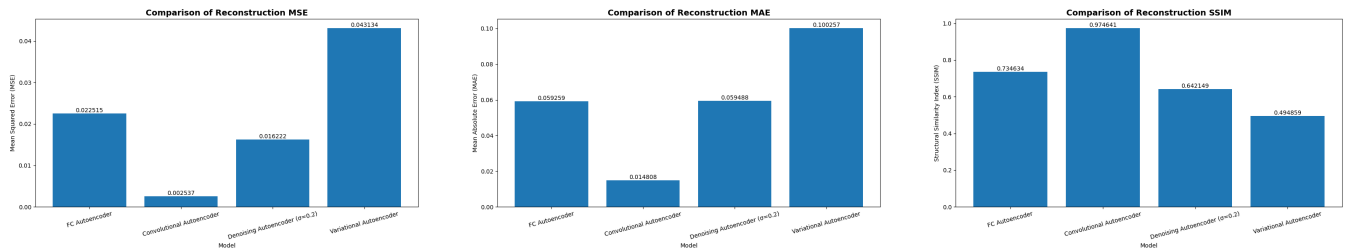


Figure 11: Final model comparison from the executed notebook: MSE, MAE and SSIM.

The final comparison highlights that the convolutional architecture retained image structure effectively with a substantially smaller parameter count than the fully connected model. The denoising model reduced corruption-related error, while the VAE traded direct reconstruction quality for a probabilistic latent space and generation capability.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	MAE	SSIM	Parameters	Time (s)
2	0.048060	0.112089	0.416966	210,196	26.57
8	0.031502	0.078545	0.624622	210,784	23.64
16	0.027183	0.069233	0.668263	211,568	22.44
32	0.028791	0.072354	0.652213	213,136	23.91

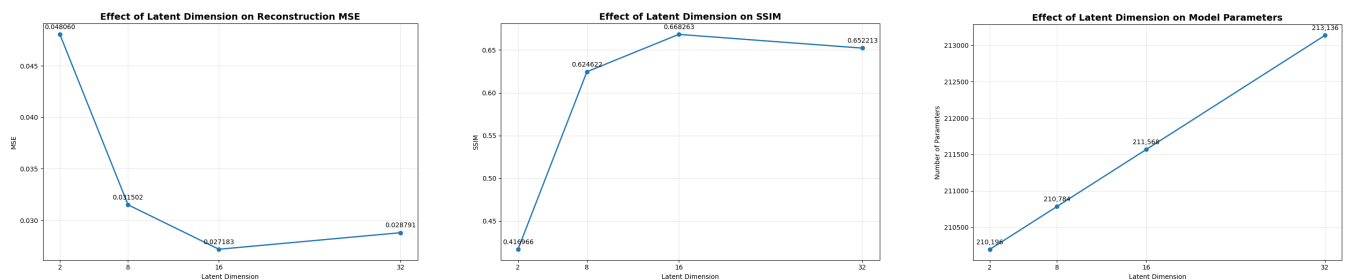


Figure 12: Latent-dimension study: reconstruction MSE, SSIM and parameter count.

Plot 10: Latent Dimension vs. Reconstruction MSE

Students should explain the trade-off between compression and reconstruction quality.

Observed result: Increasing the latent dimension from 2 to 16 improved reconstruction quality in this run: MSE fell from 0.048060 to 0.027183 and SSIM rose from 0.416966 to 0.668263. Increasing the dimension further to 32 did not improve the measured MSE or SSIM, with MSE = 0.028791 and SSIM = 0.652213. The study therefore demonstrates that additional latent capacity can help until the particular model/data configuration reaches a point where the extra dimensions do not translate into better test reconstruction.

28. Discussion Questions and Answers

The answers below are kept short and based on the experiment and notebook results.

1. What is an autoencoder?

An autoencoder learns to reconstruct its input. It has an encoder, a latent space and a decoder.

2. **What is the purpose of the encoder?**
The encoder converts the input image into a smaller latent representation.
3. **What is the purpose of the decoder?**
The decoder uses the latent representation to reconstruct the image.
4. **What is a latent representation?**
It is the compact representation of the input learned by the encoder.
5. **Why is the latent representation usually smaller than the input?**
It creates a bottleneck, so the model has to keep only the important information.
6. **What happens when the latent dimension is too small?**
Too little space can lose important information and increase reconstruction error. In the notebook, dimension 2 had MSE 0.048060.
7. **What happens when the latent dimension is increased?**
The model gets more space to represent the image. MSE improved from 0.048060 at dimension 2 to 0.027183 at dimension 16, but dimension 32 did not improve it further.
8. **Why can a convolutional autoencoder preserve spatial information more effectively?**
Convolutions work with local image regions, so they keep useful spatial patterns. The CAE test MSE was 0.002537 and SSIM was 0.974641.
9. **Differentiate an autoencoder and a denoising autoencoder.**
A normal autoencoder reconstructs the input image. A denoising autoencoder takes a noisy image and learns to reconstruct the clean image.
10. **Why is the clean image used as the target in denoising?**
Because the aim is to remove the noise and recover the original clean image.
11. **What happens when the noise level is increased?**
The image becomes more corrupted, so reconstruction becomes harder. The notebook tested noise levels 0.1, 0.2 and 0.3.
12. **Why are MSE, MAE and SSIM useful for image reconstruction?**
MSE and MAE show pixel error, while SSIM shows how well the image structure is preserved.
13. **What is a Variational Autoencoder?**
A VAE learns a probability distribution in the latent space instead of one fixed latent vector.
14. **How does a VAE differ from a conventional autoencoder?**
A normal autoencoder gives a fixed latent vector, while a VAE learns μ and $\log \sigma^2$ and samples from them.
15. **Why does a VAE learn a distribution instead of a single latent vector?**
It gives the latent space a probabilistic structure, which allows the decoder to generate new samples.
16. **What is the reparameterization trick?**
It samples the latent vector as $z = \mu + \sigma \odot \epsilon$, where ϵ comes from a standard normal distribution.
17. **Why is ϵ sampled from a standard normal distribution?**
The VAE uses $\mathcal{N}(0, I)$ as its prior, so standard-normal noise is used for sampling.
18. **What is the role of KL divergence in the VAE loss?**
It keeps the learned latent distribution close to the chosen prior. The notebook reported mean KL divergence of 6.013857.
19. **What is the prior distribution used in the VAE?**
The prior is the standard multivariate normal distribution, $p(z) = \mathcal{N}(0, I)$.
20. **Why is a two-dimensional latent space useful for visualization?**
Two latent values can be plotted directly on a 2D graph, making the latent space easier to see.
21. **What does latent-space interpolation demonstrate?**
It shows how the generated image changes when we move gradually between two latent points. The notebook used 11 interpolation steps.
22. **Why can a VAE generate new images?**
We can sample a latent vector from the learned prior and pass it through the decoder. The notebook generated 25 samples this way.
23. **Why might some VAE-generated images be unclear?**
The VAE balances reconstruction with KL regularization, so some fine details may be lost. Its test MSE was 0.043134 and SSIM was 0.494859.
24. **What is the difference between reconstruction and generation?**
Reconstruction starts from an existing image. Generation starts from a sampled latent vector.
25. **Why should test images remain unseen during training?**
It gives a fair check of how well the model works on images it did not train on. The notebook used 10,000 training and 2,000 test images.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.

2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The executed experiment demonstrates the complete pipeline on 10,000 training and 2,000 test MNIST images. The final results show clear differences among deterministic, convolutional, denoising and variational representations. The report combines quantitative evaluation with the actual notebook visualizations. Numerical performance values in this final version are taken from the supplied notebook execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>

32. GitHub Link

My Repository Link: <https://github.com/mokshithreddynarapureddy/DeepLearningLab/tree/main/Experiment-7>